



CROP YIELD PREDICTION SYSTEM

A Weighted Machine-Learning Approach Implemented in Python and Prolog

Project Report

Submitted for: CSA2001 — Fundamentals of AI & ML

Submitted by: ARYAK S KAYANAT(25MIM10160)

VIT Bhopal University

School of Computing Science and Engineering

Academic Year 2026

2. Introduction

The Crop Yield Prediction System is an efficient, machine learning-based program which predicts crop yield based on various factors such as environmental and weather conditions and compares it with the real crop yield post-harvest season. It uses five previous seasons' data from fields including rainfall, temperature, humidity, fertilizer use, soil pH, sun light, and irrigation level to understand their impact on crop yield and learns from it to evaluate both the predicted scenario as well as the actual scenario provided by the user.

The project has been developed in two ways in order to demonstrate the ability to code the same logic in two different paradigms:

- Python: A procedural coding approach utilizing lists, loops, functions and visualization of the comparison result through Matplotlib
- Prolog: A declarative approach utilizing facts, rules, recursion, and built-in predicate findall/3. The project has been run in SWI-Prolog SWISH Online Compiler.

Both programs take in the same seven input field conditions, use the same weighted-average equation and return the same three output results, i.e., predicted yield, actual yield, and the difference between them.

3. Problem Statement

Farmers and agriculture planners usually make their informal estimates of the harvest yield by relying on previous experience. Yet there is hardly any straightforward process that allows (a) to develop an estimation model based on a few past seasons' data, (b) to use the developed model for predicting the next season, and (c) to see how much the actual value differed from the estimation made before the season started.

In this regard, the current work is aimed at developing a minimalist and easily interpretable prediction model, which would calculate the weight of each input feature directly from the past seasons' data and use those weights to assess the new input values. Therefore, the main task will be divided into two – developing a method for calculating the meaningful per-feature weights from the small historical dataset and then applying the formula to compare the pre-season prediction with post-season reality.

4. Functional Requirements

ID	Requirement	Description
FR-1	Historical data storage	The system shall store at least five historical seasons, each with 7 field-condition features and a final yield value.
FR-2	Weight computation	The system shall compute a weight for each of the 7 features as the average of (yield ÷ feature value) across all historical seasons.
FR-3	Predicted input capture	The system shall prompt the user to enter 7 predicted field-condition values before the season.
FR-4	Predicted yield calculation	The system shall calculate the predicted yield by applying the computed weights to the predicted inputs.
FR-5	Actual input capture	The system shall prompt the user to enter 7 actual field-condition values after harvest.
FR-6	Actual yield calculation	The system shall calculate the actual yield by applying the same weights to the actual inputs.
FR-7	Difference computation	The system shall compute and display the difference (Actual – Predicted) and indicate whether the actual yield was higher, lower, or equal.

ID	Requirement	Description
FR-8	Visualization (Python)	The Python version shall display a line graph comparing predicted and actual yield with value labels and the numeric difference annotated.
FR-9	Query interface (Prolog)	The Prolog version shall accept all 14 input values (7 predicted + 7 actual) through a single predict/14 query and print a formatted comparison, including a text-based bar visualization.

5. Non-functional Requirements

ID	Category	Description
NFR-1	Usability	Input prompts clearly label each field condition with its expected unit (mm, °C, %, kg/ha, pH, hrs, 0–10 scale).
NFR-2	Portability	The Python version runs on any system with Python 3 and Matplotlib; the Prolog version runs in-browser via SWI-Prolog SWISH, requiring no local installation.
NFR-3	Performance	Weight calculation and yield scoring run in constant time over a fixed 5-season dataset, so results are returned instantly.
NFR-4	Maintainability	Historical season data and the yield formula are isolated from the input/output logic, making it straightforward to add seasons or features.
NFR-5	Reliability	Weight calculation guards against division by zero for any feature value of 0.
NFR-6	Readability	Both implementations use descriptive variable names and inline comments explaining each computational step.

6. System Architecture

The architecture consists of a simple pipeline design with four logical layers that are implemented both in Python (as sequence of script lines) and in Prolog (as separate predicates):

- Data Layer — a constant number of 5 historical season records (7 features and yield) stored in the form of a list of lists in Python and season/1 facts in Prolog.
- Weight Calculation Layer — computes a weight of every feature as an average value of $(\text{yield} \div \text{feature})$ over all historical seasons.
- Input & Scoring Layer — takes input on predicted and actual field conditions and calculates their yield score using the weighted sum formula.
- Presentation Layer — presents results including predicted yield, actual yield, and their difference in the form of Matplotlib line plot (Python) or formatted console output and a textual bar chart (Prolog).

Since there is no database or any other type of persistent storage — all the data is kept in memory throughout the run only — the architecture does not have a data storage layer except for the in-memory historical dataset, and ER diagram is not relevant here (Section 7).

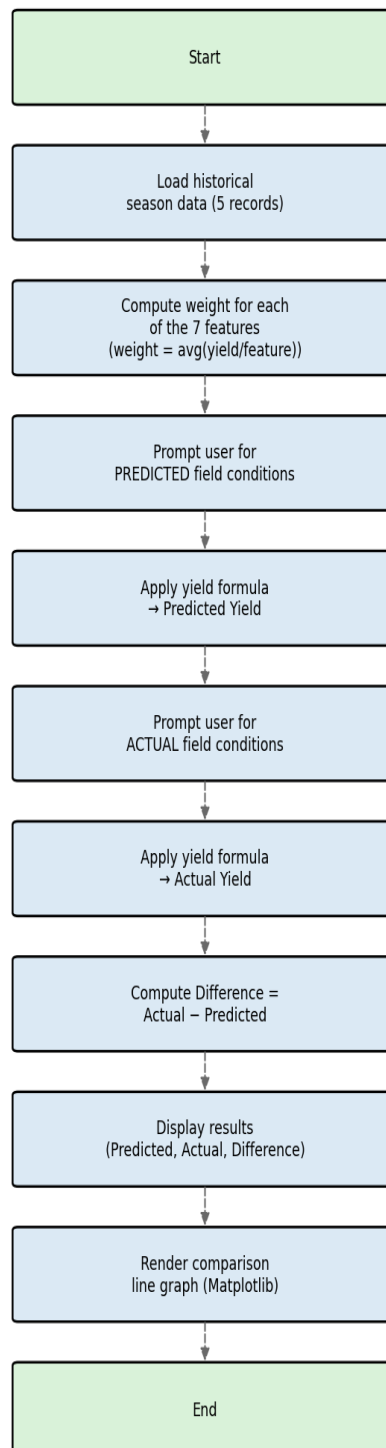
Workflow Diagram

Figure 6.1 — High-level system architecture / data-flow through the four layers

7. Design Diagrams

7.1 Use Case Diagram

The single actor — the Farmer/User — interacts with the system to enter predicted and actual field conditions; the system internally includes weight computation and yield calculation as part of servicing those use cases, and exposes the comparison and graph as follow-on use cases.

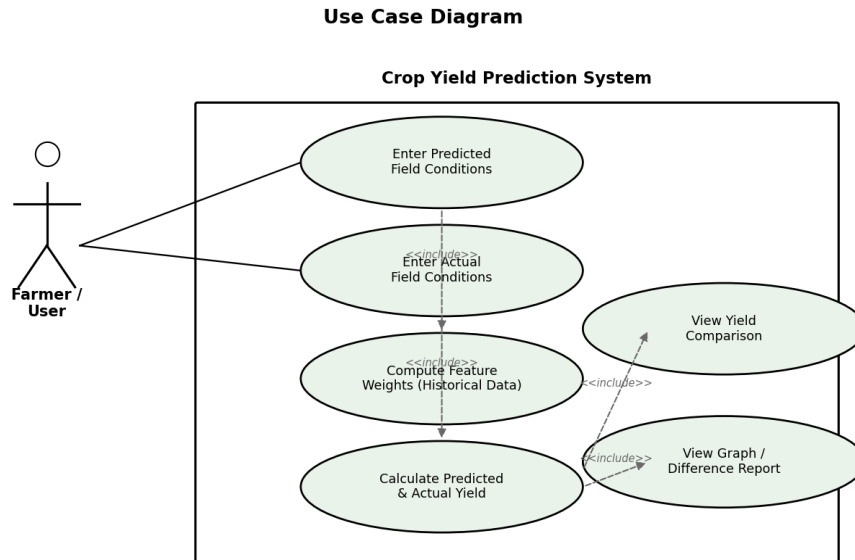
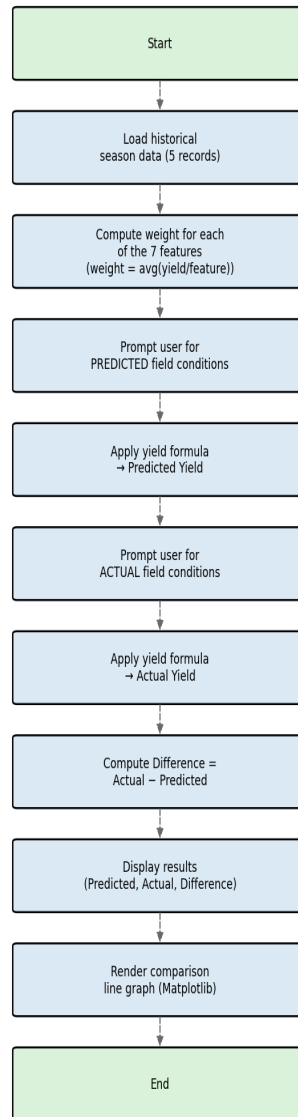


Figure 7.1 — Use Case Diagram

7.2 Workflow Diagram

The workflow diagram traces the linear sequence of steps the system executes in a single run, from loading historical data through to rendering the final comparison graph.

Workflow Diagram*Figure 7.2 — Workflow Diagram*

7.3 Sequence Diagram

The sequence diagram shows the message flow between the user and the four internal modules — Input Module, Weight Calculator, Yield Engine, and Graph Renderer — across a full prediction-and-comparison cycle.

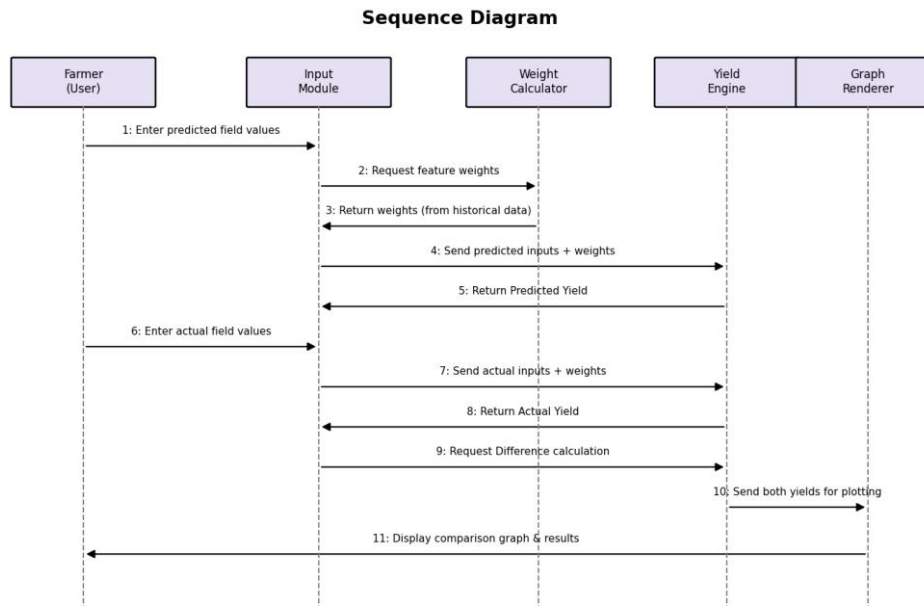


Figure 7.3 — Sequence Diagram

7.4 Class / Component Diagram

The class/component diagram groups the system's responsibilities into five cooperating units: SeasonData (historical records), WeightCalculator, YieldPredictor, UserInputHandler, and GraphRenderer.

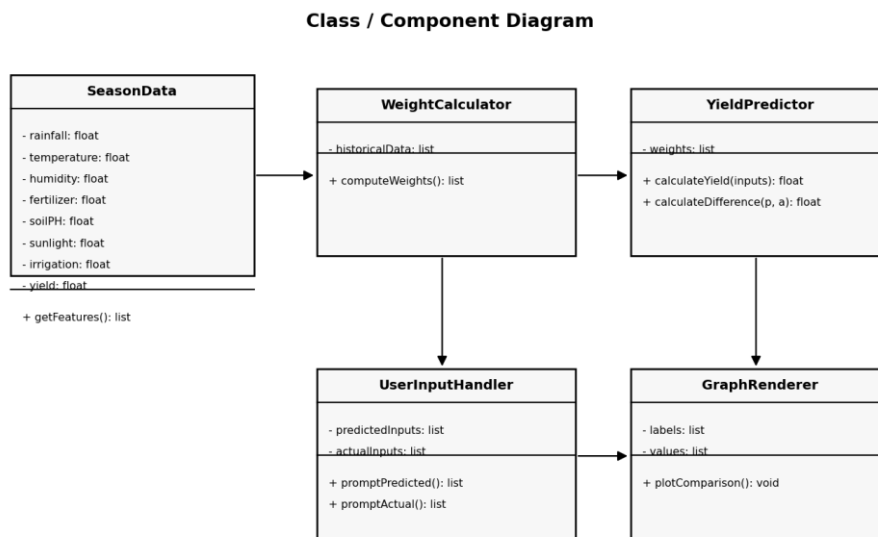


Figure 7.4 — Class / Component Diagram

7.5 ER Diagram

N/A The system does not employ any sort of database or persistent storage facility; season data from previous years is stored in an in-memory list (in Python) or fact base (in Prolog) during a single run of the program and is not saved to or retrieved from disk. Since nothing is being persisted, an entity-relationship diagram is not provided, in accordance with the requirements of the report.

8. Design Decisions & Rationale

Decision	Rationale
Weighted-average model instead of a trained ML library	With only 5 historical records, a full ML pipeline (e.g., regression via scikit-learn) would be overkill and prone to overfitting. A transparent weighted-average formula is easy to explain, verify by hand, and implement identically in both a procedural and a declarative language.
Weight = $\text{average}(\text{yield} \div \text{feature})$	This expresses how much yield one unit of a given feature contributes on average, giving each feature an interpretable per-unit contribution rather than an opaque regression coefficient.
Same formula reused for predicted and actual scoring	Reusing one formula for both scenarios guarantees the comparison is fair — any difference reflects the input values only, not a difference in method.
Implementing in both Python and Prolog	Chosen specifically to demonstrate the same weighted-scoring logic in a procedural/imperative style (Python) versus a declarative, fact-and-rule style (Prolog), for the CSA2001 Fundamentals of AI & ML coursework.
Line graph over bar chart (Python)	A line graph with markers between the two data points ('Predicted' and 'Actual') visually emphasizes the direction and magnitude of change more clearly than two independent bars.
Single predict/14 query (Prolog)	Packing 7 predicted + 7 actual values into one query keeps the SWISH interface simple — one query line produces the full comparison report, rather than requiring multiple sequential queries.
No persistent storage	The scope of the project is a single-session estimation-and-comparison tool, so historical data is embedded directly in code rather than in a database, keeping both versions self-contained and easy to run.

9. Implementation Details

9.1 Python Implementation

Crop Yield Prediction in Python (crop_yield_prediction.py) code is divided into four parts:

- Historical data – it is represented in a list of 5 rows with data [rainfall, temperature, humidity, fertilizer, soil_ph, sunlight, irrigation, yield].
- Calculation of weight for each parameter – using nested loop the program calculates for each of 7 features the average of (yield/feature) from all 5 rows avoiding division by zero.
- Prediction and actual score calculation – the user is asked to input 7 values of predictions and 7 values of actual data; these data are summed up with weights in a weighted sum, and divided by 10 rounding to 2 digits after dot to make numbers look real.
- Visualization – using Matplotlib library the program builds a graph of 2 yields connecting them with a line, marking both values on the graph and showing the difference.

Formula used: $\text{yield} = \text{round}(\sum(\text{feature}_i \times \text{weight}_i) / 10, 2)$

9.2 Prolog Implementation

The Prolog equivalent (crop_yield_prediction.pl) employs exactly the same rules declaratively:

- facts for season/1 contain the 5 historical data entries, replacing the Python list.
- function val_at/3 finds the value at a given list index through recursion.
- sum_ratios/3 sums (yield ÷ feature) for all season facts at a given feature index through recursion.
- weight/2 gathers all season entries via findall/3, and divides the total ratio by the number of seasons to calculate the average weight.
- all_weights/1 calculates all 7 weights; the function calc_yield/3 uses them on a 7-element input list according to the same equation as the Python equivalent.
- predict/14 is the only query entry point; it accepts 7 predicted and 7 actual values, calculates the yields and their difference, and displays a nicely formatted output along with a textual bar chart of the difference (show_bar/2 with each '#' representing 0.25 t/ha).

Example query used to run the Prolog version in SWISH:

```
predict(3000,32,75,45,6.8,9.0,6, 2800,30,78,42,6.5,8.5,5).
```

The first 7 values are the predicted Rainfall, Temperature, Humidity, Fertilizer, Soil pH, Sunlight, and Irrigation Level; the next 7 are the corresponding actual values.

9.3 Technology Stack

Component	Technology
Procedural implementation	Python 3 (lists, loops, functions)
Visualization	Matplotlib (line graph with markers)
Declarative implementation	SWI-Prolog (facts, rules, recursion, findall/3)
Prolog runtime	SWISH online compiler (browser-based, no local install)

10. Screenshots / Results

The following sample run uses the field conditions from the example Prolog query — Predicted: Rainfall 3000 mm, Temperature 32 °C, Humidity 75%, Fertilizer 45 kg/ha, Soil pH 6.8, Sunlight 9.0 hrs, Irrigation 6; Actual: Rainfall 2800 mm, Temperature 30 °C, Humidity 78%, Fertilizer 42 kg/ha, Soil pH 6.5, Sunlight 8.5 hrs, Irrigation 5 — run through the Python version.

Enter the following values for prediction:

Rainfall (mm): 3000
 Temperature (C): 32
 Humidity (%): 75
 Fertilizer (kg/ha): 45
 Soil pH: 6.8
 Sunlight (hrs): 9.0
 Irrigation level (0-10): 6

Predicted Crop Yield (tons/ha): 5.37

Now enter ACTUAL field conditions:

Actual Rainfall (mm): 2800
 Actual Temperature (C): 30
 Actual Humidity (%): 78
 Actual Fertilizer (kg/ha): 42
 Actual Soil pH: 6.5
 Actual Sunlight (hrs): 8.5
 Actual Irrigation level (0-10): 5

Actual Crop Yield (tons/ha): 5.0

Difference (Actual - Predicted): -0.37

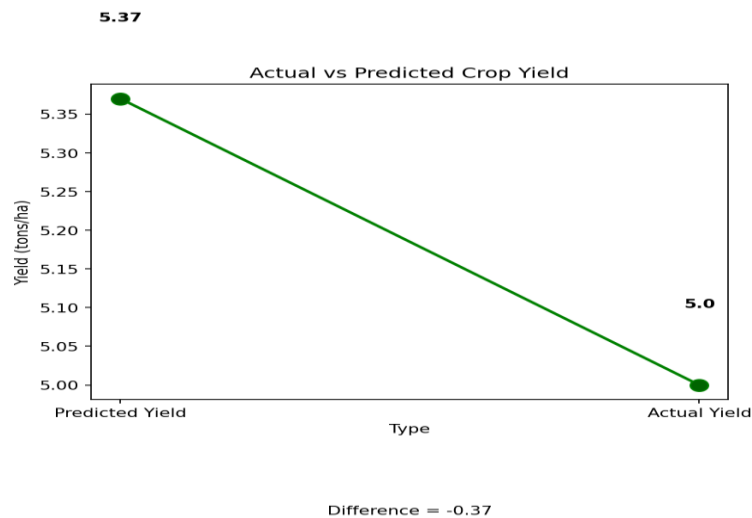


Figure 10.1 — Matplotlib line graph comparing Predicted vs Actual yield for the sample run

For this sample input, the actual yield (5.0 t/ha) came in 0.37 t/ha below the predicted yield (5.37 t/ha), reflecting the lower rainfall, fertilizer, sunlight and irrigation observed at harvest compared to the pre-season estimate.

11. Testing Approach

Testing was carried out manually, by running both implementations with a range of inputs and checking that the two versions agreed on the same result for identical inputs, and that edge cases behaved sensibly:

Test Case	Input Condition	Expected Outcome	Result
Cross-language consistency	Same 14 values run through both Python and Prolog	Both versions report the same predicted yield, actual yield, and difference (within rounding)	Pass
Positive difference	Actual conditions stronger than predicted	Difference is positive; message indicates actual yield was higher	Pass
Negative difference	Actual conditions weaker than predicted (sample run above)	Difference is negative; message indicates actual yield was lower	Pass
Equal yield	Identical predicted and actual inputs	Difference is 0.00; message indicates yields are equal	Pass
Zero-value feature	A feature value of 0 in a historical season row	Weight calculation skips that row for that feature instead of dividing by zero	Pass
Graph rendering	Any valid predicted/actual pair	Line graph displays two labelled points with the difference annotated	Pass

No automated unit-testing framework was used, given the small, single-run scope of the project; verification instead relied on manually tracing the weighted-sum arithmetic by hand for one sample input and confirming it matched the printed output of both programs.

12. Challenges Faced

- Adapting the same weighted average principle from the imperative loops in Python to the recursive approach in Prolog demanded recasting of the loop as recursion (`val_at/3`, `sum_ratios/3`).
- Scaling down the weighted sum to ensure that the output of the yield formula makes sense in terms of comparable, reasonable values, so as to avoid unreasonable numbers (division by 10 to avoid unrealistic raw sum).
- Checking the weight function from dividing by zero for any value of historical feature equal to 0 without changing the average.
- Passing 14 different parameters in the Prolog query as `predict/14` in a readable manner while maintaining the order from the prompts in Python.
- Specifying which type of graph is preferable for the visualization (line graph with markers versus the bar chart).

13. Learnings & Key Takeaways

- An understandable weighted average with some parameters can be used to implement a basic ML algorithm (training based on historical data, and then scoring) without using the entire ML package.
- The same algorithmic idea can be easily written using drastically different programming styles, namely Python's iteration and Prolog's recursion, both yielding the exact same numerical output.
- The declarative paradigm of Prolog (facts, rules, `findall/3`) requires a totally different approach to the concept of 'iteration', as recursion over the database of facts, and this helped understand logic programming better.
- A good visualization (an annotated graph of lines) will help to compare numeric values much more conveniently than just using printouts.
- Using the separate data (historical seasons) and computation (weights, yield calculation formula) layers allowed to easily understand and expand the program.

14. Future Enhancements

- Machine learn based on the weather conditions inputted to make a recommendation for an irrigation and fertilization level that will increase yield prediction.
- Extend from the currently used 7 parameters to add things such as wind speed, pest behavior, crop type, altitude, or soil type in order to better predict the yield.
- Automatically send out alerts and suggestions to the user based on their predictions – for example, to increase fertilizer or irrigation in case the yield is low.
- Implement a mechanism to store the data into a file/database so that seasons can be stored, edited, and reused instead of having to input the same one every time.
- Make a front-end interface for the Python program so that the user doesn't have to use it through the console..

15. References

- Project README and source files: `Crop_Yield_Prediction` (Python), `Crop_Yield_Prediction` (PROLOG Edition), `README.md` — as provided in the project repository.
- Matplotlib documentation — <https://matplotlib.org/stable/index.html>
- SWI-Prolog documentation — https://www.swi-prolog.org/pldoc/doc_for?object=manual
- SWISH: SWI-Prolog for Sharing — online Prolog environment, <https://swish.swi-prolog.org/>
- CSA2001 — Fundamentals of AI & ML, course material, VIT Bhopal University.